well. Likewise, we could use escape sequences to create a new line, but to do that we usually use the following `println()` function instead.

println()

The `println()` function is short for print line. After printing the specified data, it will return to the next line and start again. Its syntax is the same as `print()`:

```
Serial.println(data)
```

This function is the equivalent to either of the following groups of two statements that print the ASCII codes or escape sequence for both return and new line:

```
Serial.print(char(13));
Serial.print(char(10));
```

or

```
Serial.print('/r');
Serial.print('/n');
```

The `println()` function is a little easier to use and helps to clean up the output that we receive with the Serial Monitor. You'll often see both the `print()` and `println()` functions used in conjunction to format the output, making the text easier to read.

write()

Where the `print()` function made one assumption about the type of data that we wanted to send, the `write()` function will make the opposite assumption. Its syntax is also similar to the `print()` function.

```
Serial.write(data)
```

However in this case, the assumption is made that if we specify the value `33` we obviously mean the ASCII character code `33`, so this function will display the character `'!'`. This function effectively replaces the statements `Serial.print(33, BYTE);` or `Serial.print(byte(33));` from versions of the Arduino software prior to 1.0. As these statements are no longer valid, any time we want to display a character by calling its character code, we will need to use the `write()` function for that. Finally, the `write()` function can only send data in a single byte, limiting the values to 0–255 or the ASCII codes listed earlier.

And with that, we wrap up the last of the hardware based Serial functions that we will discuss in this chapter. There's a lot we can do with our new form of speaking to the world, but with only one serial port we are a little limited. Say we wanted to connect a new serial device, such as a reader for RFID tags that also uses serial to communicate, we would need to disconnect the device, upload our sketch, and reconnect the device to use it. Needless to say, we would not be able to fully use both the RFID reader and any other serial devices at the same time.

To get around this, we can use a new library called SoftwareSerial to create a software-based serial port on any of the Arduino's digital pins. This is hugely helpful for expanding the number of devices that we can talk to, but it's not without its limits. Our next section will look at how the Software Serial library works and use it to read data embedded in tiny little RFID tags that we can use for all sorts of things.